

Vue 2 → Vue 3 Migration Agent System

Multi-Agent AI-Powered Migration Tool

A comprehensive system for migrating Vue.js applications from version 2 to version 3 using specialized AI agents across multiple coding platforms.

Version: 1.0.0

Generated: 2026-01-16 02:21

Platforms: Claude Code, OpenCode, GitHub Copilot, Codex CLI, Gemini CLI

Table of Contents

1. Overview
 2. Architecture
 3. Supported Platforms
 4. Installation Guide
 5. Usage Instructions
 6. Migration Workflow
 7. Agent Specifications
 8. Technical Details
 9. Troubleshooting
 10. Contributing
- Appendices

1. Overview

The Vue Agent Migrator is a revolutionary multi-agent system designed to automate and streamline the migration of Vue.js applications from version 2 to version 3. This complex migration process, which traditionally requires extensive manual effort, is transformed into a guided, AI-powered experience.

Key Features

- ■ Multi-Agent architecture with specialized roles
- ■ Platform-agnostic design supporting 5 major AI coding assistants
- ■ Three-phase migration workflow with approval gates
- ■ Comprehensive project analysis and risk assessment
- ■ Automated code transformations and dependency updates
- ■ Quality validation and compliance checking
- ■ Extensible agent system for customization

2. Architecture

The system employs a sophisticated multi-agent architecture where specialized AI agents collaborate in a coordinated workflow to ensure safe and thorough Vue 2 to Vue 3 migrations.

Agent Hierarchy

Agent	Role	Responsibilities
vue-migrator	Orchestrator	Coordinates workflow, enforces phases
vue-migration-planner	Analyst	Project analysis, risk assessment, planning
vue-migration-executor	Implementer	Code migration, dependency updates
vue-migration-reviewer	Validator	Quality checks, compliance validation

3. Supported Platforms

The Vue Agent Migrator supports integration with five major AI coding platforms, providing flexibility for different development environments and preferences.

Platform	Architecture	Configuration Path	Trigger Method
Claude Code	Multi-Agent	.claude/agents/	/vue-migrate
OpenCode	Multi-Agent	.opencode/agent/	'migrate vue'
GitHub Copilot	Single-Agent	.github/copilot-instructions.md	'migrate to Vue 3'
Codex CLI	Single-Agent	.codex/AGENTS.md	'migrate to Vue 3'
Gemini CLI	Single-Agent	.gemini/GEMINI.md	'migrate to Vue 3'

4. Installation Guide

Installation is straightforward and requires only copying the appropriate agent files to your project directory. The system supports automated installation via shell scripts.

Installation Steps

1. Download or clone the vue-agent-migrator repository
2. Run the installation script for your platform:
 - Unix/macOS: `./install.sh /path/to/vue2-project`
 - Windows PowerShell: `.\install.ps1 -TargetPath 'C:\path\to\project'`
 - Windows CMD: `install.bat C:\path\to\project`
3. Select your AI coding platform from the menu
4. Verify that agent files were copied to the correct locations

5. Migration Workflow

The migration process follows a strict three-phase workflow designed to ensure safety, thoroughness, and user control throughout the migration process.

Phase 1: Planning

Description: Project analysis, risk assessment, and migration planning

Responsible Agent: vue-migration-planner

Output: Migration Analysis Document

Phase 2: Execution

Description: Actual code migration and dependency updates

Responsible Agent: vue-migration-executor

Output: Migrated codebase

Phase 3: Review

Description: Quality validation and compliance checking

Responsible Agent: vue-migration-reviewer

Output: Final Review Report

6. Technical Specifications

Detailed technical information about the migration transformations, supported patterns, and system requirements.

Migration Patterns

- Vue 2 → Vue 3 core framework upgrade
- Vue Router 3 → Vue Router 4 migration
- Vuex → Pinia state management conversion
- Options API → Composition API transformation
- Mixin → Composable pattern conversion
- Filter → Method/Computed property migration
- Event bus → Mitt library replacement
- Template syntax updates and compatibility fixes

7. Troubleshooting

Common issues and their solutions during the migration process.

Problem: Migration skips planning phase

Solution: Explicitly request analysis: 'First analyze the project and create a migration plan'

Problem: Build fails after migration

Solution: Clean install dependencies: `rm node_modules && npm install`

Problem: Components don't render correctly

Solution: Check Vue 3 syntax, verify template refs, and validate reactive data

Problem: State management not working

Solution: Ensure Pinia store is properly configured and imported